

the bottom of the job log and start with Error.

In the job log, error messages typically provide you:

1. the error
1. the file where the error occurred
1. the line number
1. the character number (where on the line it is)

For example: Error: error building site: assemble: `"/builds/gitlab-com/content-sites/handbook/content/handbook/security/security-assurance/field-security/trustcenterguide.md:1:2"`: closing tag for shortcode 'details' does not match start tag

- File: `content/handbook/security/security-assurance/field-security/trustcenterguide.md`
- Line: 1
- Character: 2

To fix markdown errors, review the message. Alternatively, review the relevant section in the markdown style guide.

For all other errors, the error message should provide the information necessary to fix it.

See the below sections for more details on how to resolves specific types of issues.

If the problem was on the main branch, you may need to rebase.

If you're unsure, you can reach out for help.

Link and anchor errors

There is a linter (Hugolint) that validates links and anchors across the handbook. If your change introduces new broken links, then the pipeline job will fail. Follow the instructions in the previous failing pipelines section for how to find the list of errors.

There are two main reasons it will fail:

1. Content added in the MR includes a broken link.
1. Content changed in the MR breaks an existing link.

Here's an example of a failed hugolint job error message when viewed in the job log:

plain

Newly broken (only in "linkcheck.json", 3 issues):

- [content/handbook/security/product-security/index.md:43]: <major> Link destination "architecture/" does not exist
- [content/handbook/security/product-security/security-architecture/index.md:269]: <major> Link destination `"/handbook/business-technology/tech-stack/panther"` does not exist
- [content/handbook/security/product-security/security-architecture/zero-trust.md:45]: <major> Link destination `"/handbook/security/corporate/systems/laptop-or-desktop-system-configuration"` does not exist

1. The error starts with the file where the broken link is present, followed by the line number.

(For example, file path - content/handbook/security/product-security/index.md, line number: 43.)

1. Next, the error indicates which link is broken. (For example, architecture/ is the broken link destination.)

1. For broken or non-existent anchor links (for example, panther):

1. Did the MR change a heading that is being linked to? If so, you'll need to update the linked anchor to match the new heading.

1. If you're linking to a heading, does it exist? Check the file in the repository instead of on the website. The links are checked pre-build, so generated content (from shortcodes and includes) don't "exist" for the link checker.

- If the page you're linking to has a large amount of generated content (such as performance indicator pages), you can add an exclusion to hugolint in the relevant configuration file.

Fixing default branch errors

MR pipelines should catch almost all errors before they are merged into the relevant repository.

The handbook projects also pull data from www-gitlab-com yml files.

If the default branch main (for public and internal handbook) or master (for www-gitlab-com) is "broken" and pipelines are failing for everyone, the root issue is most likely a data file error.

1. Check the error message.

1. If it mentions an error in building the site, or rendering something, follow the trace.

1. In the layout file, if it's failing on displaying on something that starts with site.Data.public., then it's a data file issue.

1. There are two ways to fix. Either:

1. Make the code itself more robust to deal with the types of values it may encounter.

However, not everyone knows Go and Hugo well enough to do that quickly.

If needed, create a "Handbook::Operations" "type::bug" issue in the handbook repository to document the problem

after fixing the pipeline issue.

1. Edit the data file to have values the build won't fail on.

Example: Fixing broken main on tech writing shortcode

Consider this example error:

plain

```
Error: error building site: failed to render shortcode: "/builds/gitlab-com/content-sites/handbook/content/handbook/product/ux/technical-writing/index.md:126:1": failed to render shortcode "tech-writing": failed to process shortcode: "/builds/gitlab-com/content-sites/handbook/layouts/shortcodes/tech-writing.html:16:28": execute of template failed: template: shortcodes/tech-writing.html:16:28: executing "shortcodes/tech-writing.html" at <ref page (printf "/handbook/product/categories%s-section" $section)>: error calling ref: parse
```

`"/handbook/product/categories%!(<nil>)-section": invalid URL escape "%!s"`

Following the error trace, notice that the last error with a full path and line number is:
failed to process shortcode: `"/builds/gitlab-com/content-sites/handbook/layouts/shortcodes/tech-writing.html:16:28"`.

Looking at the tech-writing shortcode,
the issue is an unexpected value in `printf "/handbook/product/categories%s-section" $section`,
which matches the last part of the error message.

From there, line 11 of the tech-writing shortcode
tells us that the data is from `site.Data.public.stages.stages "section"`.

If you have a local build of the site, you can find all the data files in the `data/public`
folder.
The relevant file (usually a `yml` file) should tell you at the top where to find the original.

If you do not have a local build, you can still likely find it in the `www-gitlab-com` data
folder.

Based on the code, you can figure out the filename. `site.Data.public.stages.stages` means it's
in `data/public` and the file is `stages.yml`.

The last parts `.stages "section"` means it's inside of `stages:` and it's pulling data from
each section: line.

You can check the most recent changes to the file, and/or compare it to when main started
failing.

In this case, an empty section: line
was the issue.

The quick and easy fix is to add text to the empty section: line, merge it, and run a new
pipeline
in the public handbook project.

In this case, the handbook code was made more robust.

Prettier is formatting markdown files

If you have prettier set up in VS Code and it is formatting the `.md` files when they are not
supposed to, check if you have Prettier set to be your default formatter with
`"editor.defaultFormatter": "esbenp.prettier-vscode"` in your user settings.

Additionally, consider using the Glob Pattern in the extension settings to specify which files
to prettify automatically.

Additional tips

For additional tips, such as how to replace strings in files, refer to the practical handbook edits examples.

Need help?

If you run into trouble editing the GitLab Handbook, help is available.

Team members, referred to as MR Buddies, are available to help you create a merge request or debug any problems you might run into while updating the GitLab Handbook. Some common questions are covered in the videos in the MR Buddies playlist. Post your request with a link in the mr-buddies Slack channel.

For general questions about the handbook, post in the handbook Slack channel.

For more serious problems, especially ones that are time sensitive or prohibiting access to important information, there is an escalation process to reach out to team members who are able to help resolve the problem.

SECTION: about/editing-handbook/edit-team-page.md

This page specifically covers how to add yourself to the team page, add your pets to the pets page, and edit the relevant entries.

For the handbook, see the editing handbook page.

Add yourself to the Team Page

We are happy to have you join our company and to include you in the team pages found on our website and in the handbook! A sync will add a basic entry for you on your third day of employment at GitLab. You are invited to personalize this entry and add more information to it. If an update is not properly reflected, verify that your Workday setting is correct. The same file is referenced for both team pages, however some of the fields that display are different. All of the available fields are documented in the team member data schema.

Ask anyone in the company for help if you need it, including the mr-buddies Slack channel. There are three ways to update the website:

1. Add your info on GitLab.com using the Web IDE (recommended)
1. Add your info on GitLab.com using the 'web interface'
1. Add your info using a Local Git clone (using the terminal and an IDE)

Choose the method below that feels most comfortable and have the following information handy:

- Name of the People Connect Team member helping you with onboarding.

- A picture of yourself for the team page

 - > Picture Requirements

 - >

- > - Crop image to a perfect square.
- > - Maximum dimension is 400 by 400 pixels.
- > - Use the JPEG (.jpg) or PNG (.png) format.
- > - Keep the file size below 100k. Minify using something like tinyjpg.com.
- > - Test image in color and black-and-white (you will add the color version).
- > - Name file yournameinlowercase with the appropriate file extension.
- Story about your background and interests. (See other team member profiles for examples.)
- Add your personal LinkedIn / Twitter / GitLab handles. When adding these handles, make sure to only include your username without any links or @ in front of them, such as LinkedIn: username. (Some incorrect examples are: LinkedIn: linkedin.com/in/username, LinkedIn: @username.)
- A relative link to your role. If your link is <https://handbook.gitlab.com/job-families/engineering/support-engineer/> use /job-families/engineering/support-engineer/. Refer to other entries for reference.

{{% alert title="Note" color="primary" %}}

For more information on setting the Export Name/Location to Team Page? opt-in mentioned below, search for the "How to: Set Team Page Export Preferences" Google doc (internal).

{{% /alert %}}

Method 1: Add your info on GitLab.com using Web IDE

1. Go to the Handbook version of the team page and find yourself.
 - Note: if you chose No in Workday for Export Name/Location to Team Page?, you will have to find yourself by your job title instead of your name. In this case, some changes like name will be overwritten by the sync.
1. Click on the avatar above your name (or job title). A modal will open.
1. In that modal, on the bottom, click the Edit this page button.
1. Our web editor will open with your team page entry opened.
1. Update your details:
 - Update your name if needed to your FirstName LastName or PreferredName LastName
 - locality should be left empty
 - country should be set to Remote
 - Verify your role
 - If your position title is incorrect or not filled in, navigate to jobfamilies.yml and use command-F (macOS) or ctrl-F (nix) to search for your job title. You can search for files in the Web IDE using command-P (macOS) or ctrl-P (nix).
 - Check that your role has a link to your job description. If not, add a link. For example, change `<a href="">Solutions Architect</a>` to `<a href="/job-families/sales/solutions-architect/">Solutions Architect</a>`.
 - Verify reportsto lists your manager using the slug value from their team page entry. The filename (without the extension) is also the slug, meaning aname.yml has a slug of aname.
 - If you are a manager, verify the reportsto of your direct reports are referring to your slug.
 - If you're currently on a borrow request, add borrow and set the to and enddate keys, such as:

```
yaml
borrow:
  to: ramya-authappan
```

enddate: 2023-09-15

- Set your current work priorities in the workpriorities field, as an array, such as:

yaml

workpriorities:

- Product Analytics
- ModelOps

- Add the filename of your profile picture, making sure to match letter case. Delete ../gitlab-logo-extra-whitespace.png, if present. The completed line should look like this: picture: yournameinlowercase.jpg.
- Add your pronouns.
- Consider adding pronunciation for your full name to help others to pronounce your name correctly (such as, Sid See-brandy for Sid Sijbrandij).
- Add your Twitter and GitLab handles without the leading @.
- Add your Mastodon account in the format mastodon.instance/@username. Basically, the profile link without the https://.
- Ensure your list of departments is accurate. Use other team members' as a reference.
- Add your specialty.
- Add your expertise. This must be formatted as HTML. An array will display incorrectly.
- Add your own story. Use other team members' stories as a reference.
- If remote work has changed your life in a meaningful way, consider adding your own remotestory, using other team members' remote stories as a reference
- Update any data that was filled in but is incorrect.

Important: Do not use the tab character, and respect the spaces between lines to avoid breaking the page format. Referenced file names/extensions are case sensitive, and a file that is not found will cause a pipeline failure. The file should end with an empty newline or it will cause a pipeline failure.

1. To upload your image, ensure that it is prepared according to the Picture Requirements.

1. Navigate to find the team folder using the path sites/uncategorized/source/images/team/. To do this, you must first notice in the left (by default) file tree that you are in a file that is within the person folder, which is within the teammembers folder, which is within the data folder. You can close folders by clicking on the · to the left of the folder name. Once you have closed the data folder, you will see the sites folder, 6 folders down. Open sites by clicking the >, then uncategorized, then source, then images, and finally team.

1. Right click on team folder, and choose Upload.

1. Select the image you want to upload, and Open.

1. If the image filename is different from what you updated your team page file previous, you need navigate back to your team page entry. You can do this by either closing the sites folder and opening data, then team members, person, and the folder containing you file; or you can notice your file tab on the top bar, and you can click on it to be taken to that file.

1. If you did the previous step, update your picture field to your filename. Delete the content that is this line after the picture: that starts with ../gitlab etc. Make sure to match the letter case of your picture file. The completed line should look like this: picture: yournameinlowercase.jpg for example.

1. Once you have finished, click the Source Control icon on the left. It should have a small

circle with a number inside of it. See point 5 of Using the new Web IDE to edit the handbook for details.

1. Add a short description of your changes in the box above the Commit and push button. An example description would be Update details to my team page entry.
1. Click the  on the right side of the Commit and push button.
1. Choose the Create new branch and commit option.
1. You will then be in the Create a new branch name and commit field. Enter your branch name, in the format of yourinitials-add-YOURNAME-to-team-page-date or similar. Example: plh-add-paulalilyherbert-to-team-page-feb06 and press Return/Enter.
1. Look for a pop-up in the bottom right, and click on Create MR. If this message disappears, click on the notification bell icon on the bottom right, and it will bring back the message.
1. Once on the Create a new merge request page, in the Description box, under the Why is this change being made? heading, explain what changes are being made and why. For this specific MR, you can enter something like: Adding my information and picture to the team page as part of onboarding tasks. For this MR, you do not need to change anything else in the description text.
1. Scroll down and Create merge request.
1. Review the Author Checklist in the description, and check off all applicable tasks.
1. Add your People Connect onboarding team member and Manager as Reviewers. On the right side, you'll see a section for reviewers, and you can add them by editing the list and searching for their names or usernames.
1. Once they have reviewed and do not request any changes, they should approve and merge the MR.

Method 2: Add your info on GitLab.com using the 'web interface'

1. Go to the GitLab.com / www-gitlab-com project.
1. Click the + under the red line near the top of the screen.
1. Click New branch.
1. For Branch name, name it something unique (it's temporary so don't worry too much about the exact name) like your initials-team-page-update-yourdepartment-the date and click Create branch. Example: hk-team-page-update-custsupport-feb06
1. Start by adding your image. Click on Repository on the left side.
1. In the file browser, navigate to sites/uncategorized/source/images/team.
1. At the top of the page click + and choose Upload file to upload your picture. Be sure to follow the picture requirements. Add a commit message in the format Add YourFirstName YourLastName to team page. Ensure the target branch is the one you previously created. Turn off the Start a new merge request with these changes toggle. Click Upload file.
1. Now you will edit your biographical information. All the bio information displayed on the Team page is pulled from a data file. Navigate to data/teammembers/persondata/FIRSTLETTEROFOURFIRSTNAME/SLUGREPLACE.yml (you are looking for a file that specifies your name or slug).
1. Navigate to the relevant yourslug.yml file. Click on Edit on the top right side of your screen, and choose Edit single file.
1. See point 5 in the above Method 1: Add your info on GitLab.com using Web IDE for the list of fields and how to fill them in.
1. After you add your information, add a commit message, and click on "Commit Changes". If you have the "create a new merge request" option checked, then it will create the merge request (MR) for you.
1. Now Create a merge request in GitLab.com with the branch that you created by clicking Create merge request button.

1. Once on the Create a new merge request page, in the Description box, under the Why is this change being made? heading, explain what changes are being made and why. For this specific MR, you can enter something like: Adding my information and picture to the team page as part of onboarding tasks. For this MR, you do not need to change anything else in the description text.

1. Click Create merge request.

1. At the upper right of the new page, click edit next to Reviewer. Set your People Connect onboarding team member and your manager as reviewer for this merge request.

Method 3: Add your info using a Local Git clone (using the terminal and an IDE)

Note: This method may take longer than other methods, because it requires git clone for a large repository.

1. Download Git, following the start using git documentation.

1. Follow the steps to create and add your SSH keys.

1. Clone the [www-gitlab-com](https://www.gitlab.com) project through your terminal, following the command line commands documentation.

1. Create and checkout a new branch for the changes you will be making.

1. Add your picture to the `sites/uncategorized/source/images/team/` directory. Be sure to follow the picture requirements.

1. Open `data/teammembers/person/FIRSTLETTEROFOURFIRSTNAME/SLUGREPLACE.yml` in your favorite editor, specifically looking for the file with your name or slug.

1. See point 5 in the above Method 1: Add your info on GitLab.com using Web IDE for the list of fields and how to fill them in.

1. Save the changes to the file in `data/teammembers/person/FIRSTLETTEROFOURFIRSTNAME/` that you just edited.

1. Optionally, to see your changes locally:

1. Manually run a command to compile the changes you just made into a file that actually populates the team page:

```
bash
cd <WWW-GITLAB-COM REPO ROOT>
bundle exec rake build:teamyml
```

1. Start a middleman development server in the uncategorized site:

```
bash
cd sites/uncategorized
NOCONTRACTS=true bundle exec middleman
```

1. Open the team page and search for your name <http://localhost:4567/company/team>

Note: Searching the handbook in your local environment yields production results, so navigate directly to the team page using the URL to see your changes.

1. Once ready, stage and commit your changes, with a comment Add details for FirstName LastName to team page.

1. Push your branch.

1. Create a Merge Request in GitLab.com with the branch that you created and assign your manager as reviewer.

Reviewing your changes

Once the MR is created, and you have a passing pipeline:

1. Look for the "View app" button in the pipeline widget on the "Overview" tab of the MR.
1. If the "View app" button does not take you to the correct page, manually visit the URL.
 - For example, use the domain of the review app <https://your-branch.about.gitlab-review.app/> and add company/team/ to form <https://your-branch.about.gitlab-review.app/company/team/> to preview your team page changes.
1. If you make changes, ensure that the latest pipeline has passed before you look at the preview. Otherwise, the latest changes will not be present.

Add your pet(s) to the Team Pets Page

Using what you learned in the steps above, consider adding your pet(s) to the Team Pets page.

The main differences are the names and locations of things.

1. The picture should be petname.jpg or petname.png with the same picture requirements.
1. Upload the picture to sites/uncategorized/source/images/team/pets.
1. Your pet information should be added to the end of the data/pets.yml file.
1. Your commit message can be similar to Adding my dog Gary to the Team Pets Page.
1. Assign your manager as the reviewer.
 - If your manager has a warning yellow triangle symbol on their avatar after adding them, they can approve, but not merge. Post in the mr-buddies Slack channel after your manager has approved with a request to get it merged.

SECTION: about/editing-handbook/practical-handbook-edits.md

This page contains instructions, tips, and historical recordings for non-technical team members on how to work handbook-first.

Content on this page is to provide additional tips as a supplement to the editing handbook page.

Please keep the basics of how to edit the handbook, including how to get help on the editing handbook page.

> Tip Use AI to edit the handbook faster, and learn about Anthropic Claude and GitLab Duo.

Before you begin in command line

Some tips may require terminal shell access on macOS/Linux. Ensure that your environment is working and that you have cloned the public handbook repository for example.

shell

git clone <https://gitlab.com/gitlab-com/content-sites/handbook.git>

Sync it. Ensure that you stash away local changed not yet committed.

```
shell
cd handbook
git stash
git checkout main
git pull
```

On macOS it is advised to use Homebrew and install the GNU tools. See this [blog post](#) for a macOS setup.

```
shell
brew install gnu-sed
```

Find files

One of the shell tools provided with macOS/Linux GNU is find. Open a terminal and run the following command in the main directory of the [www-gitlab-com](#) repository to get a list of all .md files. This matches .md as suffix.

```
shell
find . -type f -name '*.md'
```

Instead of the . you can also use a directory in the current path.

```
shell
find source/handbook -type f -name '*.md'
```

The type f specifies files, d matches for directories. When not specified, all files and directories are taken into account.

You can replace -name with -regex to do more sensitive matching, for example to match all .md and .md.erb files.

```
shell
find . -type f -regex '\.md[.erb]'
```

This can be useful to check whether a blog post was merged to the default branch:

```
shell
git checkout master
git pull
find . -type f -name 'blogpost-filename'
```

Find files and perform an action

This comes in handy when you want to print all matches with a prefix, or perform additional replace actions. The main principle is to follow the matching rules explained above, and add the `-exec` parameter.

The `exec` action should start a shell and execute a command in there. `sh -c "\;` takes care of this for every file that matches. Imagine this as a loop of sequential steps to perform the action. The last missing bit is accessing the file in the current loop iteration. This is done through the `{}` marker inside the `echo` example printing the output.

Run the command in a terminal to see how it works:

```
shell
find source/handbook/marketing -type f -name '*.md' -exec sh -c 'echo "Matched {}" \;
```

Replace strings in a file

The GNU `sed` shell command is useful to replace a defined string in a file. The `-i` flag specifies to do that inline in the same file. The `g` flag defines a global match, replacing all pattern matches.

```
shell
sed -i 's/<searchtext>/<replacementtext>/g' file.md
```

On macOS, ensure that the `gnu-sed` package is installed, and run `gsed` (instead of `sed`).

```
shell
gsed -i 's/<searchtext>/<replacementtext>/g' file.md
```

With using the `/` separator, it is necessary to escape all `/` characters in the string. You can avoid this by choosing another separator, for example `,:`

```
shell
gsed -i 's,<searchtext>,<replacementtext>,g' file.md
```

How to Find and Replace Content

```
{{< youtube id="lWBkNqXPxw8" title="How to Find and Replace Content in the Handbook using Terminal and a Code Editor" >}}
```

There are times when you need to find every instance of a word, phrase or link and searching in the handbook online search field is too cumbersome or does not return accurate results. With a code editor and your terminal, you can find and replace content easily and quickly.

Terminal

1. Navigate to the appropriate project. If you've cloned the project to your root directory, try `cd www-gitlab-com`
1. `git checkout master`
1. `git pull` or `git pull origin master` (This brings in the most recent changes from master to your local machine.)
1. `git checkout -b MYBRANCH` (Replace MYBRANCH with the name of the branch you will be using for your merge request.)
1. Keep your Terminal window open, and now open your code editor (in this example, Visual Studio Code).

Visual Studio Code

1. Go to the Search section (View -> Search).
 1. Type in the word, phrase or link you are searching for.
 1. Type in the word, phrase or link you wish to replace it with in the Replace field, under the Search field.
 1. Press the Replace All symbol at the end of the Replace field (or Option-Command-Enter).
 1. You will get a pop-up asking you to confirm that you indeed wish to Replace X occurrence(s) across X files with X ?. If correct, press Replace.
- Please note that it is possible to click on a search result to see individual changes and replace only a subset of all occurrences by clicking the replace button next to a given search result only.

1. Return to your Terminal.

Terminal

1. `git add .` (This will add all your current changes from VS Code.)
1. `git commit -m "Title of your MR"` (Ex. "Update peopleops Slack Channel to people-connect" and be sure to enter your Title within quotation marks.)
1. `git push`
1. You will get a message in terminal saying To push the current branch and set the remote as upstream, use and then there will be a sentence starting with `git push`. Copy and paste this sentence to your most current, active Terminal line, which ends in `%`.
1. You will see a line in Terminal containing `remote: <https://gitlab.com/> .....` Cut and paste the link starting with `https://` to your browser. This will take you to your Create Merge Request page. Now you can continue in your browser as you would when creating a MR in Web IDE.
1. `git checkout master` (This will put you back on master in your Terminal.)

Find and Replace a String in all (Matching) Files

Please see the following suggestions for doing a bulk find and replace.

If you rename a file/URL in one of the handbook repositories, please add a redirect.

If you rename a file/URL in the `www-gitlab-com` repository, please remember to add a redirect to `redirects.yml` by following this process.

Using a terminal

Sometimes a project, URL target or Slack channel is being renamed. You can easily search and edit with the Web IDE on GitLab.com, but for other files there is a quick and automated way required.

This method combines the find, exec, and sed tips explained above. The exec action is to use sed to perform an inline replacement of a pattern/string.

The following example is used in this MR for updating the Corporate Marketing project URL in all files.

```
shell
git checkout master
git pull origin master
```

```
git checkout -b handbook/corp-mktg-project-url
```

```
find source/handbook -type f -exec sh -c 'gsed -i "s,https://gitlab.com/gitlab-
com/marketing/corporate-marketing,https://gitlab.com/gitlab-
com/marketing/corporatemarketing/corporate-marketing,g" "{}" \;
```

```
git status
git diff
```

```
git commit -av -m "Handbook: Update corporate marketing project URL"
```

```
git push -u origin handbook/corp-mktg-project-url
```

<open URL in browser for MR>

To cut it down:

- Find and match all files in the source/handbook directory. The URL might be found in other files too.
- exec runs a sed/gsed action.
- The replacement is https://gitlab.com/gitlab-com/marketing/corporate-marketing with https://gitlab.com/gitlab-com/marketing/corporatemarketing/corporate-marketing.
- Verify the changes with git status and git diff before committing them.
- Commit, push, and create the MR from the URL.

Using Visual Studio Code

You can also do bulk find and replace operations using Visual Studio Code along with the GitLab Workflow extension for VS Code. The following steps were used in this MR for updating sub-value to operating principle.

Follow the editing the handbook locally instructions to clone the relevant repositories.

Once you've configured and installed Visual Studio Code, open the relevant directory where you've cloned the repository.

Create a new branch by:

1. Clicking on main in the bottom bar
1. Select Create new branch
1. Enter a branch name
1. Press the enter key

You can then use the Search functionality (`⌘F`) to find all files matching your desired search. Searching in Visual Studio Code allows matching case, matching whole word, or using Regular Expressions. If you click on the ellipsis (`⋮`) you also have the option to include or exclude files by pattern matching on the path (uses commas to delimit multiple includes/excludes).

You can do a Replace All across all files with the matching search term directly in the Search functionality:

1. Click on the disclosure arrow next to the Search field or enter `⌘H`
1. Enter the Search term
1. Enter the Replace term
1. Click on the Replace All button next to the Replace field or enter `⌘Enter`

Once you've made your edits use the Source Control (`⌘G`) functionality to:

1. Stage your changes by clicking on the + sign for each file, or the + sign for the entire commit
1. Provide a name to the commit in the Message field saving with a `⌘Enter`
1. Clicking on the publish or sync button

You can use the GitLab Workflow extension to manage the Merge Request (MR) once the branch has been pushed and MR created.

Remove trailing whitespaces

CI/CD linting jobs can use checks for trailing whitespaces. When detected, the checks cause the pipeline to fail, and require editing the files again. This can increase time to create and review MRs. The following tips help with removing trailing spaces in detected files faster.

shell

```
content/handbook/marketing/developer-relations-and-community/index.md:78:70 MD009/no-trailing-spaces Trailing spaces [Expected: 0 or 2; Actual: 1]
```

An IDE or the command line are common ways to fix the problem.

1. Visual Studio Code: Open the file to edit. On macOS, use the shortcut `cmd k x` to clear trailing whitespaces and `cmd s` to save the file.
1. vim: Change to the command mode (`ESC`), start a command with `:` and copy the following sequence to clear all trailing whitespaces: `%s/\s\+$//e`. Press enter, and save the file with `:wq`.

Visual Studio Code provides an editor setting to remove trailing whitespaces permanently.

Navigate to File > Preference > Settings > User > Text Editor > Files > Trim Trailing Whitespace and check the box to trim whitespaces when files are saved automatically. Please use this setting carefully, because some files contain many whitespaces to fix and can render the Merge Request change diff hard to read. This increases MR review time. It is recommended to remove and fix trailing whitespaces in a separate MR.

Bulk remove trailing whitespaces

You can replace trailing whitespaces in multiple files on the command line using GNU sed and find. The following example recursively searches the content/handbook directory for files matching the pattern .md and executes a script on each file, which replaces trailing whitespaces inline (sed -i ...).

shell

```
find content/handbook -type f -name '*.md' -exec sh -c "sed -i 's/[ \t]$/ ' '$1' '{}'" \;
```

Remove trailing whitespaces in a merge request

Sometimes the easiest way to fix one - or a few - trailing whitespace lint errors is directly in the merge request.

1. From the Changes tab in the merge request, find the file and line number mentioned in the lint error message
1. Click the speech bubble next to the line number to add a comment
1. In the comment dialog, click the insert suggestion button to copy the line into the comment dialog
1. Remove the trailing whitespace from the end of the line
1. Click Add comment now and apply the suggestion

```
{{< youtube Kfdf2VAk9sM >}}
```

```
## SECTION: acquisitions/_index.md
```

Acquisition strategy

GitLab's key goal in pursuing acquisitions is to accelerate our roadmap and offer better tools to customers more quickly. We are seeking to introduce new product categories and mature early categories faster than by building everything internally. We want to acquire strong teams that have already built great tools and products relevant for GitLab customers and have them integrate their tech into GitLab.

Additional benefits to GitLab:

1. Strengthen value prop in product categories with high growth and expected future profits
1. Improve market perception and create a PR opportunity

Acquisition approach

We are looking for companies interested in helping deliver on our vision to create a single application for the entire DevOps lifecycle. We value strong teams and technology and are looking for a combination of both that will help us accelerate our roadmap. We're looking for acquisitions which can either present a strategic value-add or create potential for significant revenue upside.

Acquisition target profile

Below is a set of general, yet not strict, characteristics of companies that are a potential fit for our acquisition process:

- Have built features or functionalities which are aligned with GitLab's long term strategy
- Engineers that meet our standard hiring guidelines
- Ideally bring a solution which accelerates our maturity in a product category or fits into the new categories on our roadmap.
- R&D team size under 30 people
- Willing to reimplement products into GitLab in Ruby and Go
- Willing to sunset old customers within 90 days or less, with an option to transition to GitLab. GitLab follows an open-core model
- Aligned to ship on the first month and iterate quickly
- Strong founders looking to join GitLab

Considering joining forces with GitLab

1. Please review our handbook; it is the central repository for how we run the company.
1. Spend time understanding if a remote culture is right for you and your team.

Why join GitLab?

1. Leverage the unique data and user-experience GitLab has as complete DevOps platform to act as a force-multiplier in helping you build the vision for your product area
1. Focus on building great technology at scale to reach millions of users and some of the largest enterprise companies. We can ensure what you made will be used by more than 100,000 organizations and millions of users.
1. Quick transaction - We move fast and aim to close the deal in 2-4 months, if you are able to drive a fast process with us
1. Upside - Financial outcome if GitLab is successful (GitLab stock)
1. Your team is in good company - GitLab is a unique place to work at with a leading product and a fascinating culture

What we offer

Acquisitions receive a compensation offer specific to that engagement, which will be evaluated as part of our acquisition process as we learn more about your company and your technology.

What happens to your current company?

1. Our goal is to integrate the acquired technology into GitLab's platform with the team behind

it, keeping the team as whole as possible.

1. We want to focus on integrating the value quickly, thus we don't want to maintain existing services/products outside of GitLab's platform as doing that will hinder the speed of integration.

Hear from the founders of our previous acquisitions

Andrew Newdigate, "Throughout Gitter's acquisition process, GitLab was an excellent partner. We started the process in mid-December, and by mid-January, the team were onboard and had joined GitLab's summit in Cancún, Mexico. Compared to previous acquisition attempts that we had been through with other companies, the professionalism and focus of the GitLab team was refreshing. It quickly became apparent to me that Sid, Paul and the team at GitLab were interested in striking a fair deal, and were as concerned about the outcome for the Gitter team as my co-founder and I were. In the year-and-half since, GitLab has continued to be a great home for Gitter and I'm incredibly proud to be part of the team."

Philippe Lafoucrière, "When GitHub announced they would provide a Security graph and alerts, we knew we would not be able to compete with them. That meant the end of the story for us and our product. It was time to find a new home for the team. We started discussions with GitLab in November 2017 and officially joined the company in mid-January 2018. The whole process was seamless, with a particular care for the team. Expectations from both sides were discussed, with mutual respect and understanding.

The founders made themselves available for us at anytime, and we had regular meetings for several months until we all estimated the situation stable enough. Onboarding a whole team at once is a challenge: the peopleops team did everything they could to help and to make them feel comfortable with their new positions. They were happy to continue on the foundations we've built over the years while being able to contribute to a greater goal. We managed to identify with the management where we could have the best impact, and provide results as soon as possible.

I don't see anything GitLab could have done differently to make this acquisition smoother. The most important value to follow in this kind of event is Trust (because you can only lose it once). GitLab, and especially its CEO, was incredibly clear and respectful during all the process and beyond."

Gemnasium: Our GitLab journey

Keys to successful acquisitions

1. Managing people's expectations and concerns
1. Successful integration execution
1. Maintain a non-biased view on target companies' value

Starting an acquisition discussion with GitLab

If you are interested in starting acquisition discussions with GitLab, please send an email to Corporate Development to connect and start the process.

Acquisition process

Learn more about our internal acquisition process and how you can prepare for an efficient

process.

Performance indicators

See our performance indicators.

Contact us

For additional information contact Corporate Development.

SECTION: acquisitions/acquisition-process/_index.md

This is a detailed view of our acquisition process. For more information about our acquisitions approach visit our acquisitions handbook.

Acquisition process

The process is comprised of five key stages:

1. Pipeline Building
1. Exploratory
1. Early Diligence
1. Confirmatory Due Diligence
1. Integration

Pipeline Building

1. Sourcing: The corporate development team closely collaborates with GitLab's product leadership to identify key areas for potential M&A. We source acquisition opportunities ("Sourced Pipeline") from:
 1. Ecosystem screen with the help of third party databases such as Crunchbase
 1. Inbound introductions from GitLab team members and industry contacts
 1. Proactive outreach to companies aligned with our vision and strategic priorities

Exploratory

1. We prioritize companies that fit with our product and acquisition priorities ("Prioritized Pipeline") and reach out to their leadership to set up intro calls.
 1. Intro calls: The purpose of this 30-min call is to learn more about the potential target company and assess if further acquisition discussions make sense. The agenda for the call is:
 1. Company overview including team, products, financials, and funding
 1. Discuss which features could accelerate GitLab's roadmap
 1. Review the GitLab acquisition process including deal terms, as appropriate (acquisitions handbook)
- TARGET TEAM: Ahead of this call please review our roadmap and outline which of your current and future product features can be implemented into GitLab's product categories. Outline a simple integration timeline for those features, considering an MVC release on the first month after joining GitLab and monthly releases following with quick iterations.

1. Share call summary Initial Acquisition Review Template (located in the Corp Dev shared drive under Acquisition Templates - GitLab internal-only) of companies that are aligned with GitLab's acquisition strategy ("Qualified Pipeline") with relevant product lead.

- Create a new, private Slack channel (format: acq-companyname) and add the internal GitLab document ("Main Acquisition Document") to the topic. Add Chief Product Officer (CPO) and the relevant product and engineering leaders to help with the initial assessment of the opportunity.

1. Product call: If the Product Champion sees a potential fit and wants to proceed, set up an initial 90-minute product call to dive into the product and tech. The call must include the Product champion and may also include Stage Leaders and specific Product Managers relevant to the call. Stage Leaders and Product Managers should keep in mind the early stage of this evaluation and attempt to think expansively about how the potential acquisition could be additive to GitLab. The agenda for the call is:

1. Product demo with highlights on key functionality and technologies
1. Concise roadmap overview with a focus on near-term plans
1. GitLab roadmap fit - discuss which features could be built into GitLab and into which product stage
1. Start the discussion about what an integration into GitLab's code base will look like

A mutual NDA will be shared by GitLab and will be required to be signed prior to the Product Call. For more details about our MNDAs and process for signing see GitLab Legal Handbook.

The Corporate Development Deal Process Manager will schedule and send calendar invites for a GitLab internal only synchronous Product Call debrief for immediately after the Product Call, or first available time slot.

1. Initial internal review:

1. Preliminary assessment of product and technology fit of the potential opportunity to GitLab's product roadmap as well as integration options into GitLab.

1. In certain instances, a hands-on product validation following the Product Call may be necessary to help inform the acquisition process and next steps. The Product Champion will select a DRI to perform a hands-on product validation. If a hands-on product validation does not occur following the Product Call, it will then occur later in the acquisition process during Early Diligence.

1. Following the hands-on product validation, the Product Champion, DRI selected for hands-on validation and Corporate Development Champion & Deal Process Manager will meet sync to debrief on findings from the hands-on product validation and align on next steps.

1. The Corporate Development Deal Process Manager will offer time to debrief on findings with the company being evaluated if the decision is to end the engagement following the hands-on product validation.

1. Before moving to early technical diligence, the Product Champion will need to confirm annual fiscal budget with the Chief Product Officer and finance team.

1. It is necessary to understand whether onboarding the target's employees can be funded through existing or planned requisitions either within the Product Champion's section, across the R&D budget or other operating expense reallocations. If not, budget must be secured by Product, with the help of Finance, before proceeding.

1. This step is intended to optimize cross-functional collaboration and prevent a case where during final approval it is determined that budget does not exist for us to onboard the target's employees, threatening the deal outcome.

Early Diligence

1. Select code name to use instead of target company name. Update Slack channel: project-codename.

1. Corporate Development Deal Process Manager to create a dedicated Project for cross-functional collaboration and issue tracking. Make a copy and rename the New Project Diligence Template to get started.

1. Form the acquisition team and add the entire team to the channel and documents.

+1. Confirm internal acquisition champion - every acquisition needs a lead champion; someone who is advocating for the acquisition, helping drive the acquisition rationale and a successful integration process. For most acquisitions that fit our approach, the champion will be a Product Section lead, at the Director+ level, accompanied by an engineering champion from the GitLab's Engineering team, at the Director+ level, respectively. For other acquisitions, champions may come from other internal functions.

1. Create a dedicated technical diligence Slack channel p-codename-technical-diligence. This channel will be the main channel for communication on technical topics with GitLab's development team.

1. Preliminary diligence - below is a list of documents to share with GitLab:

1. Financials:

1. Cap table

1. Balance sheet

1. Profit and Loss sheet

1. Cashflow statement

1. Tax returns

1. Employees:

1. Roster with: employee name, title, role, tenure, years of experience, location, salary, LinkedIn profile, programming language proficiencies, contributions to respective components and services in the codebase

1. Employee resumes

1. Employee agreements and PIAA

1. Customer list with name, monthly revenue, contract termination date and any other fields if relevant.

1. Vendor list with monthly spend

1. Asset list:

- Any assets that are needed for the business and will be part of the acquisition

- Assets excluded from the acquisition

1. Technical Bill of Materials (BOM) - a complete document which lists:

1. Repositories

1. Issue trackers / bug management systems

1. Additional (non-code) assets

1. Domain names

1. Servers

1. Dependencies

1. Database schemas

1. Data

1. Trademarks

1. Social media accounts

1. Security Reports/Documentation

1. External Security Reports

- SOC 2

- ISO 27001 Certification
- CAIQ
- External Penetration Testing Report

1. Internal Security Documentation

- Risk Management Program Documentation
- Risk Register and status of risks
- Results of security reviews the entity has performed over its current vendors

1. Early technical diligence:

1. In case the target company has open source components, the respective Dir. Engineering (dependent on GitLab stage) will start an early code review to determine: code quality, development practices, contributions, license compliance and more. That should be turned around within 2-3 business days.

1. The Corporate Development Deal Process Manager will create a new document (Project [code-name] - Technical Diligence) for the Technical Call meeting notes, separate from the main acquisition document. Future diligence findings, and all other technical diligence related note-taking of meetings (external and internal), which are engineering-centric should be recorded in this Technical Diligence document, a separate and internal Google Doc from the main acquisition document. The Technical Diligence document will be bookmarked to the Slack channel topic of p-codename-technical-diligence

1. Technical call: a hands-on product and code screen-share session (2 hours) in which the technical lead, as assigned by the respective Engineering champion, together with the respective Product champion will lead a screen-share session aimed at a hands-on validation of the product functionalities and an overview of the applications. The objectives and agenda for the call are:

1. Objectives:

1. Technically validate the functionalities and competencies of the product which have been presented throughout our process thus far

- 1. Determine the value the acquisition can potentially add to GitLab
- 1. Conclude a high level evaluation of the complexity and quality of the solution
- 1. Identify blockers, challenges for the integration

1. Agenda:

1. Cover a more technical demo of the product; cover further questions/areas which surfaced following the Product Call

- 1. Walkthrough of the architecture and the mechanisms of the product
- 1. Review the core components/services/applications as well as development practices

1. Start discussing the technical aspects of a potential path for the integration

1. Internal notes of the call should be recorded in the Technical Evaluation Google Doc

1. Access to proprietary code will not be asked at this stage. Code review will take part in the confirmatory due diligence phase.

1. Integration - a key component of post-transaction integration is the product integration strategy: prior to closing of the transaction, the GitLab product and engineering acquisition champions will formalize the integration strategy with a focus on feature sets/functionalities:

1. What we keep as-is, rewrite, discard/EOL

1. What is critical for user migration

1. Target product tiers in GitLab

1. Barriers for implementation

1. Deal Milestones:

1. We aim to set 3 milestones at 2, 4 and 6 months from joining GitLab, to provide a concise set of goals which should cover the bulk of our product interest in the target company

1. Milestones should be articulated as objectives as opposed to tasks. The structure of defining milestones should resemble that of OKRs, with each milestone having an objective and then a few key results which will be required to achieve the objective. This will help target companies focus on driving the objectives and not be tied to, and concerned with, a specific task as changes are likely to occur once integration work starts. The milestones outline the objectives to facilitate the work required in achieving the roadmap advancement the deal was identified with delivering. Each milestone should be broken down to the keys required to complete in order to achieve success for the milestone's objective.

1. Include GitLab Legal on Milestone formation discussion threads and synchronous calls to help ensure that Milestone language meets GitLab Legal's level of specificity and clarity.

1. First milestone shipped within 60 days of joining GitLab:

1. Accounting for 3 weeks of onboarding, targets will ship the first milestone 5 weeks following the end of the onboarding period

1. Critical to adopting our culture and successful future integration of the target's engineering team in GitLab

1. Allows us to show early fruits of the acquisitions soon, aligned with our value of iteration

1. Product is integrated within 6-9 months:

1. 6-9 months is an optimal timeframe which allows for incremental integration of the target's functionality, covering its entirety at best or its fundamentals at the very least, while not being overly extended. We would want to refrain from using a longer time frame as our roadmap priorities may change such that we could potentially find ourselves abandoning certain milestones, negating some or all of the rationale behind the deal.

1. Will help establish focus on both acquired target and our product team

1. Be able to complete post-closing payouts (if any earned and due) to the target's entity and shut it down as soon as practicable and in accordance with the terms of the deal documents

1. At least one milestone will focus on developing new functionality which will be based on the integration delivered in earlier milestones

1. To determine the deal ROI, the acquisition team will perform the analysis using the applicable model as well as the acquisition NPV calculator (internal GitLab document).

1. The Build vs. Buy & Integration timeline estimate delta is a key input into the deal models. As part of Early Diligence, and then later refined during Confirmatory Due Diligence, the Product and Engineering Champions and their support teams will detail and forecast both a build to parity estimate, including timeline and headcount, as well as an integration estimate forecast to deliver value to GitLab customers.

1. Direction for the Build vs. Buy & Integration estimate exercise can be found here (internal GitLab document). The Build vs. Buy & Integration template can be found here (internal GitLab document).

1. Early People-Ops review

1. The Corporate Development Deal Process Manager will create a new Slack channel topic of p-codename-people and include the People-ops Champion (VP of People-Ops) and a Talent Acquisition Lead. The People-ops Champion may request that the Corporate Development Champion and Process Manager bring in designated People-ops team-members as needed.

1. Employee roster review

1. Set your own LinkedIn profile to private mode viewing when reviewing target employee profiles. Private mode viewing will prevent target employees from being alerted to GitLab's examination of their LinkedIn profiles.

1. Compensation review - to identify any gaps and possible flags led by the HR Business

Partner

1. Founder technical interviews - founder(s) will go through two rounds of interviews to assess technical and cultural alignment.

1. The founder interview differs in style and objective of a traditional new hire interview. Interviewers will be asked to review the Interview Preparation Direction in advance of meeting with the founder(s).

1. An Application Security Review performed by GitLab's Application Security Team

1. Identifies application vulnerabilities that need to be considered by GitLab by applying a threat modeling approach to conduct the review

1. Presenting the business case for approvals (by order of occurrence):

1. Champions' approval: The acquisition team will review the business case together and approve the suggested deal. This includes approval of all the following:

1. Complete executive summary (with link to term sheet draft) including budget and headcount allocation for the acquisition

1. Complete business case templates (see Corp Dev shared drive for both - GitLab internal-only) including Deal Milestones

1. Complete term sheet (template) draft with proposed details (asset payment, retention bonuses, Deal Milestones, closing schedule, customer termination and closing conditions) filled in.

1. Functional approvals: The Corporate Development, Product and Engineering Champions will present the business case for acquisition to the CPO, CTO and CRO. They will review to approve the items listed in the Champions' approval (complete: executive summary, business case, term sheet)

1. CEO, CFO and CLO approvals: The Corporate Development, Product and Engineering Champions will present the business case for acquisition to the CEO, CFO and CLO. This meeting will also capture the explicit approval of the term sheet to start negotiations.

1. Approval of term sheet to start negotiations will be tracked in a term sheet approval issue. We don't include any financial and milestone information in the approval tracking issue for confidentiality reasons.

1. Term Sheet:

1. Once the terms to start negotiations have been approved, the Corporate Development Deal Champion will reach out to the target company to share the offer and term sheet.

1. Once an agreement on terms with the target has been reached, the term sheet (with any changes) will be brought forward for approval from: CLO, CFO, CEO (in that order). These approvals will be captured in the term sheet approval issue.

1. Once all approvals have been obtained, the Corporate Development Champion will stage the term sheet for signing on DocuSign for target CEO and GitLab CEO (in that order). Add CLO, CFO, and CPO on Cc on the agreement.

1. Approval tracking will be tracked on the term sheet approval issue mentioned earlier. Any changes to previously-approved terms need to be reviewed and approved once more by the following: Product champion, Engineering champion, CPO, CTO, CLO, CFO, CEO. The changes should be referenced on the term sheet approval tracking issue before seeking approvals.

Confirmatory Due Diligence

1. To kick-off the Due Diligence stage, the Corporate Development Deal Process Manager emails the target company CEO with the following clarifications and information requests:

1. All employees and their profiles will be reviewed by the GitLab team

1. The employees who will be invited for interviews will go through GitLab's standard interview process

1. Key employees who were interviewed during the Early Diligence stage may go through further interview rounds as determined by the GitLab team to qualify for a role at GitLab

1. All employees must identify an open vacancy at GitLab which they think best matches their professional profile. This will be shared in a spreadsheet gathered by the target's CEO.

1. The Corporate Development Deal Process Manager will create an engagement debrief and lessons learned document and share it with the team for on-demand capturing of insights.

1. Document who is read-in on the project and work with GitLab Legal to circulate a Read-in Acknowledgement form to sign and return.

1. Complete Technical diligence

1. Complete financial diligence

1. Legal diligence - Once both the technical and the financial diligence have been completed and signed off by the Engineering champion and Finance acquisition team member, respectively, the Corporate Development Deal Process Manager will contact legal to start the legal diligence.

Legal will tag the relevant owners for each of the diligence tasks in the (template diligence table (see Corp Dev shared drive - GitLab internal-only) in the main acquisition doc.

1. The progress of the diligence will be synced on a regular stand-up call with the acquisition team

1. The Corporate Development Champion and the legal lead negotiate the definitive deal documentation with the target company CEO and legal team

1. Key employee retention conversations - to increase the success rate of the deal and integration plan, we may request to conduct retention conversations for the key technical employees identified. The key technical employees are those identified as critical to the success of the acquisition, the proposed integration plan and the future of the team at GitLab post integration.

1. Final review and approval:

1. Conclusive call - Final internal review call with the acquisition team to recap the acquisition as a whole, review the acquisition agreement and present a final recommendation. This meeting will also capture the explicit approval of the acquisition agreement from the CLO, CFO and CEO. Approvals from the call as well as additional required approvals will be tracked using the definitive agreement approval issue template.

1. BoD approval:

1. Legal will facilitate the final deal approval from GitLab's Board of Directors

1. A board package will be shared with the BoD members before requesting their approval. It should include:

1. Executive summary email:

1. Will be sent to the BoD members by the CEO

1. The Corporate Development Champion and Deal Process Manager will create a 1-2 page executive summary email message. It should include a high level coverage of the following topics:

1. Deal details and rationale (including roadmap acceleration, projected revenue gain)

1. Expertise of team joining

1. Key terms for the APA and any additional costs

1. Diligence conducted: legal, tax, financial, technical, people

1. Known risks, indemnification conditions as well as representations and warranties

1. Link to final version of APA

1. The Corporate Development Champion will coordinate the signing and closing of the deal

Integration

The Corporate Development team is responsible for overseeing and facilitating the post-closing integration, working closely with Legal, Product, Engineering, People, Finance, and other GitLab divisions as appropriate. The DRI is the Senior Director Corporate Development.

The integration process is outlined in our acquisition integration page.

Terminated Acquisition Engagements

1. If the acquisition has advanced past the Technical Call, conduct a retrospective:
 1. Create a retrospective issue (GitLab internal-only example)
 1. Set time for an optional live sync
 1. Solicit feedback on all related acquisition channels
 1. Once sync has completed, continue with the steps below.
1. The Corporate Development Deal Process Manager will archive all slack channels associated with the acquisition.
 1. In accordance with the signed NDA, the GitLab Corporate Development Champion and Deal Process Manager, and all other deal participants in possession of preliminary and confirmatory deal diligence materials, will make best effort to remove the materials from GitLab's internal drives and repositories.

Acquisition Team

The primary acquisition team is designed as a compact unit, and will consist of the following GitLab functional team members:

1. Corporate Development Champion - VP of Corporate Development
 1. Corporate Development Deal Process Manager - Corporate Development Manager
1. Product Champion - Product Section Leader (reporting to the Chief Product Officer)
 1. Product Manager
1. Engineering Champion - Dir. Engineering
 1. Engineering team member
1. VP, People Operations
1. Legal, Corporate lead

The primary acquisition team will engage with other functions and roles as needed:

1. VP Finance and Business Technology
1. VP Tax
1. Principal Accounting Officer
1. People Business Partner
1. VP Talent Acquisition
1. VP of Security

To assign the product manager, after the product call or as soon as it's clear which product category the features will be implemented into, contact the category product director for the assignment.

To assign the engineering team member, contact the engineering manager of the relevant category for assignment.

Acquisition Team Responsibilities

Function	Role
Deliverables	
-----	-----
----	-----
Corporate Development	1. Main POC for acquired team 1. Identify potential areas for integration 1. Create case for acquisition and customer transition story 3. Integration 1. Business case with deal structure
Product	1. Outline current product features to be implemented into GitLab 1. Outline potential future functionalities to be built into GitLab after the integration period 1. Product integration strategy
Engineering	1. Technical diligence 1. Code quality review 1. Integration strategy validation - feasibility and timeline
Finance	1. Lead financial diligence 2. Validate business case and work with tax team to validate deal structure
Legal	1. Review entity, assets a